Replicating Interference-Free Real-Time Experiments on Ubuntu and openEuler

Alessandro Claudio Damo
University of Padua
Email: alessandroclaudio.damo@studenti.unipd.it

***Abstract*—This work presents the experimental replication of selected studies described in the paper *"Interference-Free Operating System: A 6 Years' Experience in Mitigating Cross-Core Interference in Linux"* (RTSS 2024), with the goal of assessing their validity across different hardware platforms and operating systems. In particular, the core isolation mechanisms listed in Table III of the paper were systematically applied on three kernels: Linux "Vanilla" 5.10, LinuxRT 5.10 and openEuler 22.03 (based on Linux 5.10), while carefully documenting procedures, challenges, and practical limitations. Subsequently, experiments were conducted to measure maximum latency (*cyclictest*), latency distribution (*cyclictest* and *oslat*), task-set schedulability and communication latency in ROS 2. The results show that openEuler, while not completely eliminating cross-core interference, achieves lower latencies and more stable performance compared to Linux "Vanilla", and results better but more similar to the RT kernel. The analysis further highlights that, despite the intrinsic limitations of Linux, the adoption of appropriate isolation mechanisms and specific kernel patches can significantly mitigate the impact of interference. The contribution of this work lies not only in the empirical confirmation of the paper's claims, but also in providing a practical guide for researchers and developers interested in replicating or extending these experiments.**

I. Introduction

In this introductory chapter, we present the objectives of our work, outlining both the rationale behind the study and the specific goals that guide the following experimental analysis.

A. Context of the Paper

With the advent of multicore systems, real-time applications have benefited from increased computational power and, at least in theory, from the ability to meet tighter timing constraints. However, this led to a phenomenon that must be taken into account, namely *core interference*.

Core interference is the phenomenon that occurs between different cores of a multicore processor when they share common hardware resources. Such sharing can cause the activities executed on one core to introduce **delays, jitter, or performance degradation** on other cores, even if those cores are dedicated to independent or real-time tasks.

Indeed, this phenomenon prevents Linux from fully isolating cores dedicated to real-time (RT) tasks from other system activities: even when cores are supposedly isolated, they may still experience interference that leads to jitter and delays on the isolated real-time tasks. In the following section, we will examine examples of the types of interference events that generate jitter and delays on isolated cores in Linux.

Through the contributions presented by the authors of the paper under review (*"Interference-free Operating System: A 6 Years' Experience in Mitigating Cross-Core Interference in Linux"*), this phenomenon was effectively mitigated, resulting in a substantial improvement compared to the system's behavior prior to these modifications.

B. Objectives of This Work

The objectives of this work are threefold:

1) To thoroughly understand the paper and identify the theoretical causes of cross-core interference, as well as the modifications proposed by the authors to mitigate the problem.
2) To investigate and demonstrate how to apply the proposed isolation mechanisms in order to obtain cores truly dedicated to real-time tasks (as described in Table III of the paper).
3) To replicate and illustrate the replication of some of the key experiments presented in the paper—particularly those in Sections V-B and V-C—using different hardware and operating systems, with the aim of empirically validating the claims of the original work.

C. Scope

In this work, we did not replicate all the experiments and the complete theoretical framework presented in the paper; instead, we will focus on:

- **Addressed**: isolation techniques, measurements with `cyclictest/oslat`, schedulability with fixed priority, measurements with ROS 2.
- **Not addressed**: industrial evaluations with CFS, complex schedulability analyses.

D. Achieved Contributions

- **Systematic application and procedural description of isolation mechanisms** outlined in Table III of the *Interference-Free Operating System* pa-

per, implemented on "Vanilla" Linux, Linux RT and openEuler.
- **Experimental replication** of several key experiments from the paper (latency with *cyclictest* and *oslat*, latency distribution, task-set schedulability, ROS 2 communication tests), conducted on hardware different from that of the original authors.
- **Documented experience**: a collection of procedures, commands, observations, and encountered challenges in applying the isolation mechanisms, providing a practical reference for researchers and practitioners aiming to reproduce or extend this work.

E. Experimental Setup

In this paragraph, we introduce the environment in which some of the paper's experiments were replicated:

- **Hardware**: Intel i7-3770, 4 physical cores (8 logical).
- **Operating Systems**: Ubuntu 22.04 with RT kernel 5.10, openEuler 22.03 (with kernel based on "vanilla" Linux 5.10), Ubuntu with Vanilla Linux 5.10.
- **Tools**: `perf`, `stress-ng`, `cyclictest`, `oslat`, `rt-tests`, ROS 2.

II. Background and Related Work: Understanding the Paper

This chapter provides a summary of the key concepts presented in the paper, together with the theoretical background required to understand the subsequent experiments and the paper's objectives.

A. Cross-Core Interference

Cross-core interference occurs when execution on one core introduces variable delays in the execution of another core, without direct interaction, thereby undermining **temporal predictability** in real-time systems.

According to the paper, this phenomenon can compromise the operating system's ability to isolate **real-time (RT) tasks**. By *OS-level isolation* we refer to the mechanisms already provided in Linux to dedicate certain cores exclusively to RT activities (e.g., `isolcpus`, `nohz_full`, `irqaffinity`, etc.).

In practice, this means that even cores reserved for real-time may still experience disturbances from other processors in the system, resulting in delayed task **activations**, unintended **preemption** of high-priority tasks, and increased blocking times for RT activities.

These issues arise from the activation of system threads or non-essential tasks on the isolated cores, thereby interrupting the execution of high-priority tasks. Specific examples of these cases will be presented in Section 2e.

B. Main Contributions of the Paper

- Highlighting that **the OS itself is a primary source of temporal interference**.
- Sharing industrial experience in identifying and fixing cross-core interference bugs in Linux.
- Extracting **general mitigation principles** and lessons for future research.
- Evaluating improvements through **schedulability analysis** and **real-world case studies**.

C. Demonstrating the Existence of Cross-Core Interference

The paper aims to demonstrate that the current Linux kernel can be a source of temporal interference (i.e., introduce unpredictable delays) and that the isolation mechanisms provided by the operating system are insufficient.

To show this, the authors partitioned their multicore processor into two groups:

- P_r: cores dedicated to real-time (RT) tasks.
- P_n: cores dedicated to non-real-time workloads.

All available isolation mechanisms (as listed in Table III of the paper) were enabled.

The underlying assumptions were:

- Tasks running on P_r meet their real-time constraints.
- P_n does not communicate with P_r.
- P_n may be arbitrarily overloaded.

Despite these conditions, the authors observed interference events and increased latencies using *ftrace*. They concluded that, even when all isolation mechanisms are applied, Linux itself introduces additional activity on isolated cores. The subsequent experiments quantify this interference and evaluate how far the proposed patches improve the situation.

D. Core Isolation Mechanisms

Another important contribution of the paper is its detailed explanation of the core-isolation mechanisms currently available in Linux. These mechanisms were applied both in the original experiments and in our own replications.

Some mechanisms discussed in the paper but not considered in this work are related to NUMA architectures, which are not present in our system and are therefore not relevant to our experimental analysis.

Kernel Compilation Options: The kernel must be compiled with options that minimize interruptions and background activity on cores reserved for RT tasks.

Kernel Boot Parameters: In the GRUB configuration, the following parameters are used:

- `isolcpus=Pr_cores_list`: removes selected cores from the general scheduler.
- `nohz_full=Pr_core_list`: eliminates periodic scheduling ticks.
- `rcu_nocbs=Pr_core_list`: offloads RCU callbacks to system cores.
- `irqaffinity=Pn_core_list`: ensures interrupts are handled only by system cores.
- `pcie_aspm=off`: disables PCIe power-saving features that may introduce latency.

Disable `irqbalance`: `irqbalance` is a daemon that automatically distributes IRQs across CPUs. Disabling it ensures that interrupts remain confined to manually assigned system cores.

IRQ Affinity on System Cores: Explicitly binding hardware interrupts (IRQs) to system cores is essential to prevent disturbances on isolated cores.

Disable Transparent Huge Pages (THP): THP aggregates 4KB pages into 2MB pages. Allocation and splitting of huge pages occur asynchronously and unpredictably and must therefore be disabled in RT or interference-free systems.

Disable `memory.move_charge_at_immigrate` (cgroups v1): This feature migrates memory pages when a task moves between cgroups, introducing non-deterministic memory movements that must be disabled.

Disable Kernel Same-page Merging (KSM): KSM merges identical memory pages used by multiple processes (e.g., in virtual machines or containers). Its background scanning introduces unpredictable delays and must be turned off.

Move `kswapd` to System Cores: `kswapd` is the kernel thread responsible for swapping and page reclaim. It must be pinned to system cores to prevent disturbances on RT cores.

Restrict `pcrypt` to System Cores: The `pcrypt` module performs parallel cryptographic operations (AES, SHA, etc.). When enabled, it may schedule parallel tasks across multiple cores and must therefore be restricted to system cores.

Restrict Workqueue `cpumask` to System Cores: Many asynchronous kernel activities are handled through workqueues executed by `kworker` threads. By default, they can run on any core, including isolated ones, introducing interference. Their execution must be restricted to system cores.

Disable Timer Migration: Timer migration allows the kernel to move active timers across cores for load balancing and energy efficiency. It must be disabled to prevent timers from firing on isolated cores.

RCU Non-Preemptible (`rcu_normal=1`): RCU (Read-Copy-Update) uses asynchronous callbacks which, in preemptible mode, may interrupt RT tasks. Enabling non-preemptible RCU enforces a more predictable behavior and reduces jitter.

Disable RT Runtime/Period Limits: By default, Linux enforces CPU runtime limits for `SCHED_FIFO` and `SCHED_RR` tasks. In interference-free setups, where cores are fully dedicated to RT tasks, these limits are unnecessary and must be disabled.

Disable Swap: Kernel swap activity is inherently non-deterministic and unacceptable in RT or interference-free systems. Swap must be completely disabled.

E. Experience in Mitigating Cross-Core Interference

The paper summarizes six years of Linux bug fixing and patching aimed at minimizing cross-core interference (34 bugs fixed with 40 patches). The authors classified the main bugs and described their resolutions.

Scheduling and Task Placement Bugs: **a) Workqueue management bug**

Context: Workqueues manage asynchronous kernel tasks executed by worker threads.

Problem: Linux may wake worker threads on all cores, including isolated ones, via inter-processor interrupts (IPIs), causing unwanted context switches and jitter.

Solution: A worker thread should only be awakened if its queue contains pending items, and non-isolated cores must never wake isolated cores.

b) Incoherent task migration bug

Context: Tasks can be migrated across cores.

Problem: Migration may violate isolation constraints, moving a main thread but leaving child threads behind on isolated cores.

Solution: Migration must be coherent across the entire thread group.

Resource Management and Sharing Bugs: **a) ASID isolation bug across cores**

Context: ASIDs allow multiple address spaces to coexist in the TLB.

Problem: ASIDs are globally shared; if consumed by non-isolated cores, RT tasks may incur additional TLB flushes.

Solution: Partition the ASID space between isolated and non-isolated cores.

b) Device queue flush bug

Context: Devices maintain per-core queues.

Problem: Removing a device triggers flushing on all cores, including isolated ones.

Solution: Flush only when necessary and warn if triggered by non-isolated cores.

Concurrency and Coordination Bugs: **a) Locking bug on *jiffies***

Problem: Isolated cores may spin while accessing shared timer variables.

Solution: Minimize critical sections and remove redundant locks.

b) Distributed coordination bug (system statistics)

Problem: Periodic global statistics updates activate work on all cores, including isolated ones.

Solution: Restrict statistics updates to the relevant partition.

F. Lessons Learned

Despite significant improvements, the authors acknowledge that complete elimination of cross-core interference is currently impossible in Linux.

Linux lacks a well-designed mechanism to enforce strict isolation across cores. The system is composed of numerous independent subsystems, each developed with different goals and without a unified isolation strategy.

Existing mechanisms such as `isolcpus`, `nohz_full`, and `irqaffinity` are independent and not integrated into a coherent framework. Engineers must manually combine multiple configurations, without a single rule guaranteeing full isolation.

Furthermore, shared hardware resources (caches, memory buses, I/O devices) introduce unavoidable temporal interference, even when software-level isolation is enforced.

Finally, limited academic involvement and the absence of strong theoretical foundations hinder the development of systematic and formally validated solutions for core isolation in Linux. Most existing approaches are empirical and lack standardized evaluation metrics, reducing generalizability across heterogeneous platforms.

Overall, without a deep theoretical and architectural redesign, fully eliminating cross-core interference remains an open challenge.

III. Experimental Methodology and Results

In this chapter, we present the execution of the various experiments and the results we obtained.

A. Introduction

The experiments presented in this chapter are conducted on a system already **isolated** as described in Table III, and in many cases with **interference** generated by the system cores, as detailed in each paragraph following the **methodology** of the paper (which, in turn, is grounded in the theoretical framework summarized in section 2).

As in the original paper, the experiments were carried out on the following **kernels**:

- RT kernel 5.10
- openEuler 22.03 (based on Vanilla 5.10)
- Vanilla Linux 5.10

The choice of these slightly older kernels is motivated by the fact that many of the authors' modifications have since been **merged upstream** into the Linux source code. Consequently, the more recent versions of the kernel already include all the changes introduced by openEuler. By relying on these older versions, not only do we work with kernels that are comparable, but we can also perform a complete and rigorous evaluation of the modifications.

Implementation details are discussed in Section 7.

B. Interference events on isolated cores measured with Ftrace

As discussed in Section 2.5, examples of interference events correspond to undesired activities that occur on an isolated core despite the kernel isolation mechanisms. As shown in Section 2a, such events can delay task activation due to additional latency introduced by interference, erroneously preempt high-priority tasks, increase the blocking time of real-time activities, and worsen task schedulability.

The reference paper classifies these events into three main categories:

TABLE I: Number of interference events recorded on isolated cores in openEuler, Vanilla Linux, and Linux RT (10-second ftrace trace; non-isolated cores under stress). Lower values indicate fewer cross-core interference events.

	openEuler	**Vanilla**	**Linux RT**
Context switch	21	32	26
IPI	6	198	14
TLB flush	0	0	0

- **IPI handling**: inter-processor interrupts that, if received on an isolated core, introduce unwanted disturbance.
- **Context switch**: undesired context switches caused by workqueues, migrations, or global locks.
- **TLB flush**: invalidations of the TLB cache that result in performance degradation and increased latency.

In this section, we aim to quantify the number of cross-core interference events occurring on the three different systems and to evaluate how effective the modifications proposed by the authors of the reference paper are. Unlike the original paper, we replicated the measurements also on Vanilla Linux and Linux RT.

Using **Ftrace**, we performed a 10-second analysis and event count on a single **isolated** core; meanwhile, the remaining cores were heavily loaded with programs generating interference (thread creation, memory allocations, network operations, and memory stress).

From the data that we can see from [Table I], it is clear that **openEuler significantly reduces interference events** compared to Vanilla Linux and Linux RT.

- **IPIs** are almost absent on openEuler (6 events), while on Vanilla Linux they reach very high values (198), highlighting that core isolation is much more effective in openEuler.
- **Undesired context switches** are also lower in openEuler (21) compared to Vanilla Linux (32) and slightly better than Linux RT (26), confirming that scheduling and workqueue handling on isolated cores are more stable.
- **TLB flushes** are absent in all cases.

Overall, the experiment shows that **openEuler guarantees more effective core isolation**, reducing interference events and unwanted disturbances due to context switches and IPIs.

C. Measuring the maximum latency of a real-time task under interference

The **maximum latency** of a real-time task represents the worst-case response time observed between the activation of the thread and its actual wake-up. It is a crucial parameter for real-time systems: if the maximum latency exceeds the safety margin of the deadline, the task fails. A lower maximum latency also implies higher schedulability,

TABLE II: Maximum latency of a single real-time task on an isolated core with cross-core interference, measured with `cyclictest` (1,000,000 iterations). Lower values indicate lower latency and higher schedulability.

Configuration	openEuler	Vanilla	Linux RT
Max latency (μs)	72	153	89

as it allows a larger number of real-time tasks to meet their deadlines.

In this section, we quantify the impact of cross-core interference on maximum latency and evaluate how effectively the fixes proposed by the authors improve this characteristic.

We therefore used *cyclictest* (with 1,000,000 iterations) to measure the **maximum latency** on a single isolated core, while the non-isolated cores were stressed with programs generating heavy interference (thread creation, memory allocations, network operations, and memory stress).

The paper does not provide detailed information about this experiment beyond the use of *cyclictest*. It appears to have been conducted on all 24 isolated cores, but no information is given regarding the number of threads. To resolve this ambiguity, we considered **a single isolated core running a single thread**. This choice is also more consistent with the schedulability calculations, which operate at the level of individual cores.

From what we can see from [Table II] The results clearly show that **openEuler guarantees significantly lower maximum latency** compared to Vanilla Linux and slightly better results than Linux RT. The maximum latency on openEuler is 72 μs, less than half of the 153 μs observed on Vanilla Linux and lower than Linux RT, confirming that the proposed solutions effectively reduce cross-core interference and actively decrease the maximum latency of real-time tasks.

D. How cross-core interference affects the maximum latency of a periodic real-time task

In this section, we measure and quantify how cross-core interference affects the maximum activation latency of periodic real-time tasks with period T. The maximum activation latency is a crucial parameter, as it indicates whether the system can **meet regular and repeated deadlines** even in the presence of interference, and therefore achieve higher schedulability.

We evaluate the impact of cross-core interference on maximum latency by varying the activation period T, thus testing the phenomenon under different **temporal load** conditions.

To verify this, the authors use *cyclictest*, which measures the wake-up jitter of a periodic thread. The experiment is conducted by partitioning the cores: a single isolated core runs a high-priority real-time *cyclictest* thread, while

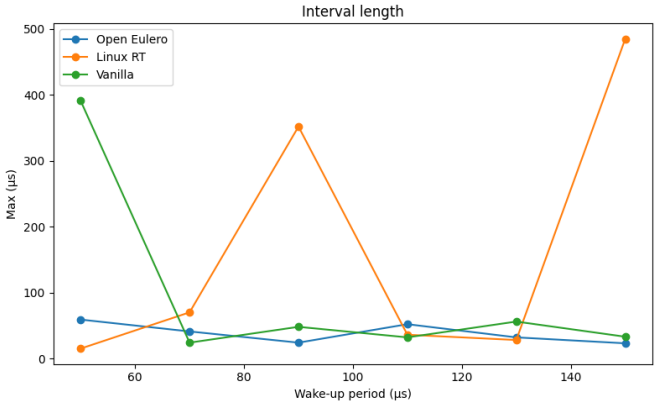

Fig. 1: Measured maximum latency of `cyclictest` for a periodic real-time task running on an isolated core. The X-axis represents the activation period in microseconds, while the Y-axis shows the maximum observed latency.

the activation period is varied from 50 to 150 μs. The system cores are loaded with programs generating heavy interference.

The results [Fig 1] clearly show that **the effect of cross-core interference on real-time task activation latency varies significantly across the tested systems**.

- **openEuler** maintains relatively low and stable maximum latencies across almost all periods, providing robustness under different temporal loads.
- **Vanilla Linux** shows latency values comparable to openEuler for most periods, except for the most demanding case (50 μs), where latency increases dramatically (391 μs).
- **Linux RT** exhibits inconsistent behavior: in some periods latencies remain low (15–36 μs), while in others they increase drastically (352 μs at 90 μs, 485 μs at 150 μs), indicating that under certain temporal loads the system may fail to meet real-time constraints.

E. How cross-core interference affects the maximum latency of a real-time task set with an increasing number of threads

In this section, we again focus on the maximum activation latency of real-time threads, but this time as a function of the number of threads and therefore of the amount of **concurrency load**.

This experiment investigates how cross-core interference impacts **activation latency** as the number of real-time threads running on isolated cores increases. To this end, the authors again use *cyclictest*, which measures the wake-up jitter of periodic threads.

The experiment is conducted by running a variable number of high-priority real-time threads (from 1 to 48), all with a 50 μs period, on the four isolated cores, while the non-isolated cores are stressed with programs generating

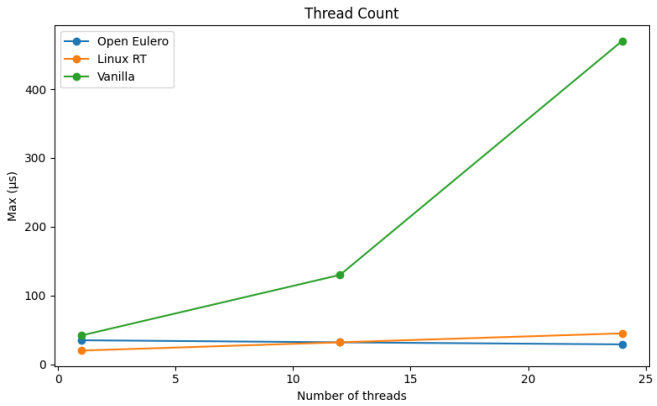

Fig. 2: Measured maximum latency of `cyclictest` for real-time tasks running on isolated cores with a 50 μs period. The X-axis represents the number of threads running simultaneously, while the Y-axis shows the maximum observed latency. Each line corresponds to a different Linux variant. For 48 threads, the test diverged and is therefore not shown.

heavy interference (thread creation, memory allocations, network operations, and memory stress). Maximum latency is measured for each configuration.

The results [Fig 2] clearly show that:

- **Increasing the number of real-time threads on isolated cores increases the maximum latency**: as the concurrency load grows on a fixed number of cores, activation latency inevitably increases.
- **openEuler** exhibits relatively low and stable latencies up to 24 threads, demonstrating good isolation behavior and effective resource management. Although the test diverges at 48 threads, the stability observed up to 24 threads indicates that the system can efficiently handle a moderate number of simultaneous real-time tasks.
- **Vanilla Linux** shows a drastic increase in latency already at 24 threads (470 μs), confirming that core isolation is insufficient and that interference from non-isolated cores rapidly degrades real-time performance.
- **Linux RT** exhibits intermediate latencies (45 μs at 24 threads), but with a slightly faster increase compared to openEuler.
- At 48 threads, the measurements diverge for all systems: beyond a certain threshold, the load and interference make results excessive, highlighting the practical hardware limits of the experimental platform.

Overall, this experiment confirms that **openEuler provides the best predictability and stability** under an increasing number of real-time tasks, whereas Linux RT and Vanilla Linux show significant performance degradation under cross-core interference.

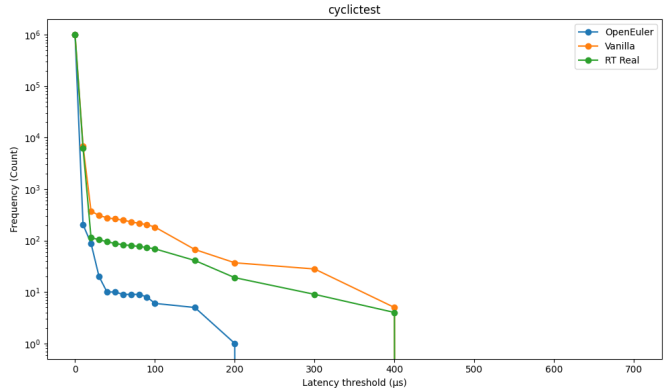

Fig. 3: Latency distribution of `cyclictest`. The X-axis represents latency thresholds in microseconds, while the Y-axis shows the number of occurrences over 1,000,000 iterations. A more compact distribution indicates higher predictability and lower jitter.

F. How cross-core interference affects the distribution of activation latencies with cyclictest

In this experiment, we focus on the **distribution of activation latencies** of a periodic real-time task, i.e., how often a thread is awakened later than its programmed activation time. This metric is fundamental in real-time systems because not only the maximum latency matters, but also how frequently latencies exceed certain thresholds: high frequencies of delays indicate a **less predictable system that is more prone to deadline violations**.

This experiment aims to show how cross-core interference influences the **distribution of activation latencies** of a real-time task. To verify this, we again use *cyclictest*, which measures the wake-up jitter of periodic threads.

In the original paper, the experiment was performed with 24 threads on 24 isolated cores. To preserve proportionality, we replicated it on four isolated cores, with each core running a high-priority real-time thread with a 50 μs period. As before, the remaining system cores were loaded with programs generating heavy interference (thread creation, memory allocations, network operations, and memory stress).

In this case, we measure how often the activation latency exceeds a given threshold over 1,000,000 iterations.

The results [Fig 3] clearly show that **the distribution of real-time task latencies is strongly affected by cross-core interference**:

- **openEuler** keeps most activations below very low thresholds: only a few events exceed 50 μs and almost none exceed 200 μs, indicating highly predictable behavior. The latency distribution is the most compact and concentrated at low values.
- **Vanilla Linux** exhibits a much higher frequency of latencies above 10–20 μs, with extreme values up to 400 μs, confirming that isolated cores experience sig-

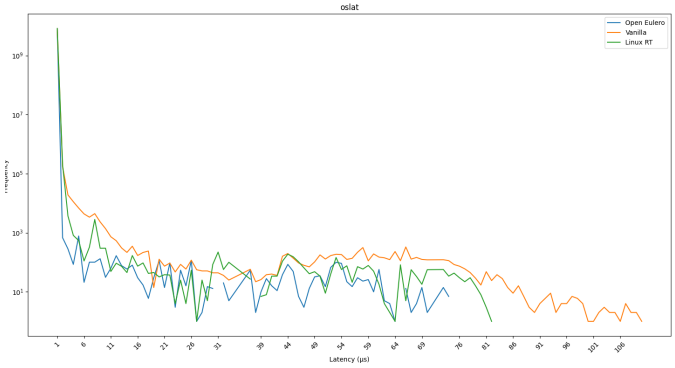

Fig. 4: Latency distribution measured with `oslat` for a high-priority busy-loop real-time task running on isolated cores.

nificant interference from non-isolated cores, resulting in a wider and less predictable latency distribution.

- **Linux RT** performs better than Vanilla Linux but worse than openEuler: frequent latencies accumulate above 10–20 μs and values up to 150–200 μs occur regularly, indicating a more dispersed behavior compared to openEuler.

In summary, this experiment demonstrates that **openEuler guarantees a much more compact and predictable latency distribution**, drastically reducing the probability of large delays. In contrast, Vanilla Linux and Linux RT are more susceptible to high jitter caused by cross-core interference.

G. How cross-core interference affects the distribution of latencies with oslat in busy-loop execution

In this experiment, we focus on the **distribution of latencies of a real-time task executing continuously in a busy loop**, i.e., on the variability of execution times of a high-priority thread that does not rely on periodic wake-ups but instead runs a continuous loop. This characteristic is fundamental in real-time systems because it highlights how well the system can execute continuous tasks with predictability and without jitter.

Unlike the other experiments, this measurement was carried out **without additional stress** generated by the non-isolated cores.

This experiment aims to show how cross-core interference influences the distribution of latencies when the real-time task does not merely perform periodic wake-ups but instead executes a continuous busy loop. To investigate this, the authors employ *oslat*, a tool that measures the latency of a high-priority busy-loop thread. The experiment is conducted on a configuration with four isolated cores, each running by default one measurement thread, whereas in the original paper the experiment was performed on 24 isolated cores.

The results from [Fig 4] highlight that **openEuler guarantees superior predictability even for real-time tasks executing a continuous busy loop**, without additional interference:

- Most executions on **openEuler** are concentrated in the lowest latency buckets (1–3 μs), with very few events exceeding 10 μs, indicating extremely stable and predictable behavior.
- **Vanilla Linux** exhibits a much more dispersed distribution: although many events fall within low-latency buckets, numerous occurrences extend to tens of microseconds, revealing significantly higher jitter.
- **Linux RT** shows intermediate behavior, with more events beyond the first buckets compared to openEuler, but less dispersion than Vanilla Linux, indicating that core isolation is not fully effective even in the absence of external interference.

In summary, this experiment confirms that **openEuler is able to maintain minimal and consistent latencies even for continuous real-time tasks**, whereas Linux RT and Vanilla Linux are more susceptible to jitter.

H. How cross-core interference affects schedulability under fixed-priority scheduling

In this section, we analyze how cross-core interference impacts the **schedulability** of a set of real-time tasks under **Fixed-Priority (FP) scheduling**, i.e., the ability of the system to complete all tasks within their respective deadlines. Schedulability is a crucial metric in real-time systems, as it indicates whether the system can correctly handle workloads without deadline violations, even in the presence of interference from other cores.

In this case, the authors of the paper do not execute the tasks directly on the system, but instead rely on the *SchedCAT* framework, using as input the jitter values previously measured with *cyclictest* (maximum latency experiment).

In contrast to the paper, we employ a homemade implementation that fulfills all the specified requirements:

- 500 task sets with 40 tasks each, per utilization level
- Execution costs derived from utilization and period
- Periods uniformly distributed in [10, 100] ms
- Implicit deadlines equal to periods
- All tasks assigned a fixed release jitter taken from the maximum latency
- Task priorities assigned according to the Rate Monotonic policy

This procedure was applied to progressively increasing total utilizations $U \in \{0.50, 0.60, 0.70, 0.80, 0.85, 0.90, 0.95, 1.00\}$ and repeated for all three kernels under evaluation.

The schedulability analysis of each task set was carried out on a single core, unlike the paper where the evaluation considered 20 cores. This choice simplifies the schedulability calculation and ensures consistency with the jitter value taken from the maximum latency experiment, which was also conducted on a single core.

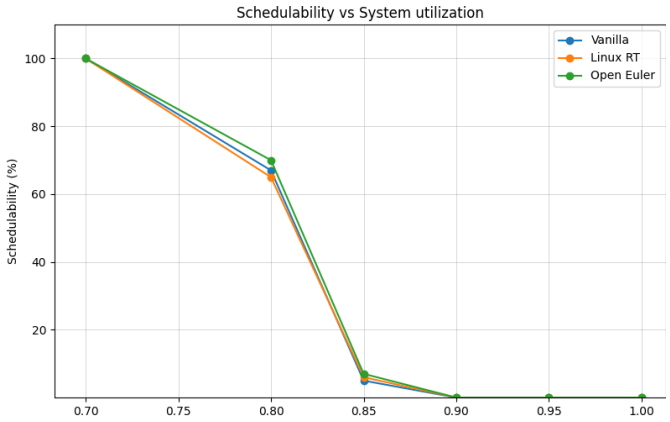

Fig. 5: Schedulability of real-time task sets under Fixed-Priority scheduling as total utilization increases.

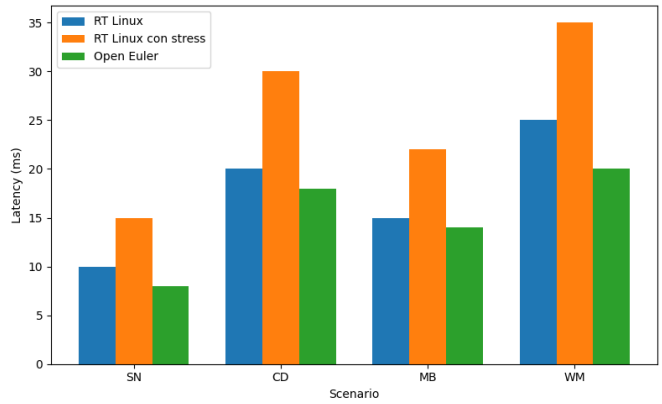

Fig. 6: Maximum ROS2 communication latency under different interference conditions.

To achieve this, we used **Response-Time Analysis (RTA) for fixed-priority scheduling with release jitter**:

This is the classic extension of Joseph and Pandya's RTA to account for jitter (and blocking), later developed in the Audsley–Burns–Tindell–Wellings line of work in the early 1990s. We solve each task's response time using a fixed-point iterative approach, from which schedulability of the task set can be determined. Blocking time is ignored, since the paper explicitly states:

> "All tasks are independent of each other without any shared resources among them."

By definition, this implies null blocking time.

The data reported in [Fig 5] show how the **schedulability of real-time tasks under Fixed-Priority scheduling** is affected by cross-core interference:

- Up to $U = 0.70$, all kernels guarantee 100% schedulability, indicating that for moderate workloads interference does not compromise task execution.
- At $U = 0.80$, openEuler maintains a slightly higher percentage of schedulable task sets (71.6%) compared to Linux RT (69.2%) and Vanilla Linux (66.4%), already showing improved behavior at this utilization level.
- For higher loads ($U \geq 0.85$), schedulability drops sharply for all kernels, with openEuler still slightly better but overall very low (6.2%), highlighting the practical limits of real-time task management under heavy congestion.
- At $U \geq 0.90$, no system guarantees schedulability.

Overall, these results indicate that openEuler achieves better schedulability, although the margin is not large, and that cross-core interference does not excessively impact Fixed-Priority scheduling on a single core.

I. How cross-core interference affects ROS2 communication latency

In this experiment, we analyze the **maximum communication latency** between nodes in a complex real-time system based on *ROS2*. Communication latency is a crucial indicator of predictability in distributed real-time systems: high or highly variable latencies can compromise node synchronization and the correct execution of real-time tasks in robotic or industrial applications.

This experiment investigates how cross-core interference affects the maximum communication latency within a **complex real-time** system based on ROS2. To this end, the authors employ the *iRobot ROS2 Performance* evaluation framework, which measures the maximum message transmission latency between ROS nodes.

The experiment is conducted across different node topologies (*sierra_nevada*, *cedar*, *mont_blanc*, and *white_mountain*), each characterized by a distinct number of nodes and workload intensity. Real-time ROS2 processes are pinned to isolated cores with maximum priority, while non-isolated cores are loaded with workloads generating heavy interference (thread creation, memory allocations, I/O operations, and memory stress). The experiment was run for approximately two minutes, recording the maximum communication latency in three scenarios: RT-Linux without interference, RT-Linux with interference, and openEuler with interference.

The data reported in [Fig 6] show how **communication latency in ROS2 real-time systems** is affected by cross-core interference:

- **openEuler** maintains lower communication latencies compared to RT-Linux under stress, with values between 721 ms and 740 ms, demonstrating greater **resilience** even in complex scenarios with heavy load on non-isolated cores. This enables correct and stable execution with reduced risk in robotic or industrial applications.

- **RT-Linux under stress** exhibits significantly higher latencies (851–904 ms), highlighting that cross-core interference heavily degrades real-time middleware performance, worsening end-to-end communication between nodes and potentially compromising application correctness.
- **RT-Linux without stress** shows intermediate latency values (750–798 ms), confirming that the observed differences are mainly due to interference generated by non-isolated cores rather than the middleware itself.

IV. Discussion

Starting with the first experiment, the **Ftrace analysis** clearly shows that openEuler exhibits the fewest interference events during the 10-second tracing window with background interference from the other cores. The numbers are not dramatically different from those of **Linux RT**, but both are better than the **Vanilla kernel**. As reported in the paper, despite the applied modifications, interference events are not completely eliminated in openEuler: **a small number of unavoidable events remain**, tied to fundamental aspects of kernel operation.

This is consistent with the results shown in Figure 5 of the paper (where we unfortunately did not capture a rare TLB flush event), which also demonstrates that interference events persist in openEuler. The reason why, despite core isolation and the use of openEuler with *interference-free* mechanisms, some events (e.g., residual IPIs) still occur is clearly explained in **Lesson 3: Synchronization Mechanism Suitable for Isolation** of the paper.

The authors state:

> "In existing Linux, it is almost impossible to completely eliminate synchronization between cores. Thus, we can hardly avoid the interference introduced by synchronization. Our current effort alleviates the interference, but it cannot fully address the interference because we did not choose to completely redesign the synchronization system."

In short, in current Linux **it is impossible to fully eliminate inter-core synchronization**, and therefore some interference is inevitable. The authors' intervention greatly mitigated the issue, but did not resolve it completely, precisely because they chose not to redesign the synchronization system from scratch in order to avoid compromising fundamental kernel logic. This also connects to other reasons highlighted in **Lesson 1** of the paper:

> "It is impossible to fix all cross-core interference bugs in Linux. The key reason is that Linux does not have a well-designed mechanism to enforce isolation between cores."

It should also be noted that the **limitation observed during isolation** did not cause major problems in the subsequent experiments, which remained consistent with the results in the paper. This issue likely introduced additional interference and worsened the results, but in a **uniform way across all three kernels**, such that proportionality among them was preserved.

Regarding the **maximum latency calculation**, the data are consistent not only with the paper but also with the Ftrace event counts: **openEuler shows lower latency and values similar to Linux RT**, while Vanilla Linux performs significantly worse. The relative differences between the kernels align with the paper, although the baseline latency achieved in openEuler is drastically different (12 μs versus 72 μs). This discrepancy is certainly due to hardware differences, yet the overall consistency remains acceptable. As discussed later, the improvement in maximum latency also has direct consequences for schedulability.

The results of [Fig.1] [Figure 6a for the reference paper], however, are less aligned. Based on the critical data, we expected decreasing latency values across all three kernels, since increasing the wake-up period should physiologically reduce latency. This behavior occurs only in **openEuler**, partially in **Vanilla Linux**, and very poorly in **Linux RT**, where the data exhibit a veritable *rollercoaster*. According to the paper (and the experiments before and after), Linux RT was expected to clearly outperform Vanilla Linux. However, despite Vanilla Linux's poor performance at a 50 μs wake-up period, its trend is more stable than that of Linux RT. While this unexpected behavior limits the significance of the experiment, it still provides an interesting observation: openEuler follows an excellent trend as the temporal load varies, outperforming the other two kernels and aligning with theoretical expectations.

Moving to [Fig.2] [Figure 6b for the reference paper], a clarification is needed. As explained earlier, the analysis was limited to **24 threads**, since the paper conducted the experiment on 24 isolated cores and 48 threads simply represented twice that number. In our case, with only 4 isolated cores, the results with 48 threads diverged excessively, reaching values close to seconds in all three kernels, making graphical representation impractical. Nevertheless, the 48-thread experiments were still executed for completeness, and openEuler continued to perform best among the three.

Considering the more reliable data, Figure 2 shows the expected increasing trends for Vanilla Linux and Linux RT. For openEuler, the trend even appears to decrease, which is clearly impossible; however, the differences are so small that they instead indicate stability and predictability under varying temporal load. As expected, this stability is lost at 48 threads, where all results diverge, although still less severely for openEuler. In this case, Linux RT performs better than Vanilla Linux and approaches openEuler, yielding results that are overall consistent with the paper.

[Fig.3] [Figure 7a for the reference paper] shows results that align very well with the paper: openEuler clearly performs best in terms of frequency distribution, never

exceeding 200 μs of latency, with most values concentrated near zero. Linux RT and Vanilla Linux behave similarly to each other, both reaching maximum latencies of approximately 400 μs and exhibiting wider distributions. The only discrepancy lies in the absolute values, which can be reasonably explained by hardware differences, even though the experimental setup is equivalent (24 cores and 24 threads in the paper, 4 cores and 4 threads in this work).

The same applies to [Fig.4] [Figure 7b for the reference paper], conducted with a different tool (*oslat*). The results are highly consistent and further confirm the validity of the experiment.

Regarding the schedulability tests in[Fig.5] [Figure 8b for the reference paper], since the maximum latency values from Table 4.2 were used as input, similarly coherent results were expected and indeed obtained, despite relying on a homemade implementation. The **best performer is openEuler**, which theoretically allows tasks to be scheduled efficiently on a single core up to a utilization of 0.8, but collapses already at 0.85. The other kernels follow the expected trend: Linux RT performs very close to openEuler, while Vanilla Linux is slightly worse.

Compared to the paper, the differences among kernels are narrower. This is due both to the values in Section 4.2, which are already quite similar, and to the fact that schedulability was analyzed on a **single core**, whereas the paper evaluated schedulability on **20 cores**, where differences naturally become more pronounced.

Finally, the ROS2 experiments also show results consistent with the paper [Fig 6]. OpenEuler under stress slightly outperforms RT-Linux without stress, which in turn performs significantly better than RT-Linux under stress. The absolute latency values differ substantially from those reported in the paper (milliseconds instead of microseconds), likely due to differences in latency computation. Additionally, the simplest ROS2 topology, *cedar*, unexpectedly resulted in the second-worst performance. Nevertheless, the proportional consistency with the paper remains satisfactory, allowing us to conclude that openEuler ensures greater system stability and a lower risk of errors.

V. Conclusion

In conclusion, this work confirms that the current **Linux kernel is indeed a source of temporal interference**. This phenomenon leads to degraded performance in terms of latency, schedulability, and system predictability, and demonstrates that the isolation mechanisms provided by the operating system are insufficient due to cross-core interference.

Despite extensive **bug-fixing efforts**, the phenomenon still persists, although in an attenuated form. Through the presented experiments, we quantitatively assess the extent to which this interference has been **mitigated**. The absolute performance values are worse than those reported in the paper, likely due to the significantly less powerful hardware used in this study. Slower processors and memory systems increase execution and data access times, leading to higher latency under identical workloads. Moreover, the paper benefited from up to 20 isolated cores, whereas our experiments were limited to 4 isolated cores, making worse absolute results physically expected.

Nevertheless, the **proportional trends remain consistent**. In line with the findings of the paper and despite hardware differences, **openEuler consistently emerges as the best-performing system** across all experiments, although its performance is closer to Linux RT than reported by the original authors.

Under cross-core interference, openEuler consistently exhibits lower latencies, which translate into improved schedulability, as lower maximum latency allows a larger number of real-time tasks to meet their deadlines. These improvements are also observed in periodic tasks under varying temporal and concurrency loads. Furthermore, the latency distributions are more compact and centered around minimum values, indicating a more predictable system that is less prone to deadline violations.

Even in practical and industrial scenarios such as ROS2-based systems, openEuler demonstrates superior performance, showing greater **resilience** in complex workloads with heavy stress on non-isolated cores, thereby enabling correct and stable execution with reduced risk in robotic and industrial applications.

Finally, the preliminary and exploratory event-counting experiments further suggest a relationship between the number of interference events and performance degradation: more interference events correspond to worse results, with Linux RT and openEuler performing relatively close to each other and both significantly better than Vanilla Linux. While these results are insufficient to draw definitive conclusions, they provide an interesting and theoretically consistent indication that warrants further investigation.

References

[1] H. Kim, M. Kim, H. Choi, H. Cho, J. Lee, and H. Kim, "Interference-free Operating System: A 6 Years' Experience in Mitigating Cross-Core Interference in Linux," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2024.

[2] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[3] A. Burns, A. J. Wellings, N. C. Audsley, and A. Tindell, "Fixed Priority Scheduling with Deferred Preemption," *IEEE Transactions on Software Engineering*, vol. 19, no. 4, pp. 331–339, 1993.

[4] B. Andersson and E. Tovar, "Schedulability Analysis of Global Multiprocessor Scheduling Algorithms," in *Proc. 12th Euromicro Conference on Real-Time Systems (ECRTS)*, 2000.

[5] L. Abeni, L. Palopoli, G. Lipari, J. Regehr, *et al.*, "SchedCAT: A Scheduling Theory Toolkit," [Online]. Available: https://github.com/brandenburg/schedcat

[6] OpenEuler Project, "openEuler 22.03 LTS Documentation," [Online]. Available: https://www.openeuler.org/en/

[7] ROS 2, "Robot Operating System 2 Documentation," [Online]. Available: https://docs.ros.org/en/

[8] J. Corbet, A. Rubini, and G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed. O'Reilly Media, 2005.

[9] Open Source Tools Used:

- **rt-tests** (cyclictest, oslat). [Online]. Available: https://git.kernel.org/pub/scm/utils/rt-tests/rt-tests.git
- **stress-ng**. [Online]. Available: https://kernel.ubuntu.com/git/cking/stress-ng.git